

Graph Traversals Continued : $\rightarrow$ (No single ans)

DFS Traversal : $\rightarrow$ Depth First Search

Adjacency List : $\rightarrow$

```

1  $\rightarrow$  2, 3
2  $\rightarrow$  1, 4, 5
3  $\rightarrow$  1
4  $\rightarrow$  2, 5
5  $\rightarrow$  2, 4

```

Recursion

1, 2, 4, 5, 3

visit 2 :

```

1 2 3 4 5
T T T T T

```

src

```

1
/ \
2   3
/ \ / \
4 5 7 8

```

done

Adj - List

```

1 - 2, 3
2 - 1, 4, 5
3 - 1
4 - 2, 5
5 - 2
6 - 2
7 - 3, 8
8 - 7

```

visit 2

```

1 2 3 4 5 6 7 8
T T T T T T T T

```

1, 2, 5, 6, 3, 7, 8, 4

src

```

1
/ \
2   3
/ \
4   5

```

Adj - List : $\rightarrow$

```

1  $\rightarrow$  2, 3
2  $\rightarrow$  1, 4, 5
3  $\rightarrow$  1
4  $\rightarrow$  2, 5
5  $\rightarrow$  2, 4

```

o/p : 1, 2, 4, 5, 3

Graph Algorithms : $\rightarrow$ Placement Related : $\rightarrow$

Topological Sort Algorithm : $\rightarrow$

(Linear Ordering) (DAU)

Linear ordering of vertices such that if there is an edge from node u to node v , $u \rightarrow v$;

" u always appears before v " in the linear ordering.

Topological Sort (DFS) : $\rightarrow$ $u \rightarrow v$

u before v

o/p ① 4 5 2 3 1 0

o/p ② 5 4 2 3 1 0

DAU $\rightarrow$ Directed Acyclic Graph

edges

5 $\rightarrow$ 0

5 $\rightarrow$ 2

2 $\rightarrow$ 3

3 $\rightarrow$ 1

4 $\rightarrow$ 0

4 $\rightarrow$ 1

After dfs (node) ends push to stack & mark true.

visited array

0 1 2 3 4 5

dfs(0)

dfs(1)

dfs(2)

dfs(3)

dfs(4)

dfs(5)

5 4 2 3 1 0

stack

0, 1, 2, 3, 4

* Component wise traversal $\rightarrow$ for loop (0-5)

Topo Sort BFS : $\rightarrow$ (Kahn's Algorithm) **

Steps : $\rightarrow$ o/p : 4 5 0 2 3 1

① Insert all nodes with indegree 0 into the queue.

② Take out node from queue and reduce indegree of adjacent nodes.

New concept : Indegree : $\rightarrow$ The number of incoming edges to a particular node.

Indegree Array

0 1 2 3 4 5

2 2 1 1 1 0

0 1 2 3 4 5

Topo Sort BFS Kahn's Algorithm for Cycle Detection

Directed Cyclic Graph

Topo Sort o/p : 1

TS size $\neq V$

indegree

0 2 1 1 1

1 2 3 4 5

cycle FFF

if topo sort == No of nodes $\rightarrow$ No cycle

if topo sort $\neq$ No of nodes $\rightarrow$ Cycle

Dijkstra's Algorithm : $\rightarrow$ (Shortest Path)

(Single Source Shortest Path Algorithm)

src

distance array : $\rightarrow$

0 4 4 7 5 10

0 1 2 3 4 5

Adjacency List : $\rightarrow$

Pair (N, W) (Node, Weight)

0 $\rightarrow$ {1, 4} {2, 4}

1 $\rightarrow$ {0, 4} {2, 2}

2 $\rightarrow$ {0, 4} {1, 2} {3, 3}

3 $\rightarrow$ {4, 1} {5, 2}

4 $\rightarrow$ {2, 1} {5, 3}

5 $\rightarrow$ {2, 2} {3, 2} {4, 3}

Time Complexity : $\rightarrow$ $O(\log V)$

edges

Nodes

PS $\leftarrow$ min-heap

dist, node

Why does the Dijkstra's Algorithm not work for negative weights : $\rightarrow$

* Bellman Ford (-ve)

* Floyd Warshall (-ve)

adjacency list

Node, Weight

1 {2, -2}

2 {1, -2}

distance array

1 2

1 2

PS min-heap

dist, node

* Minimum Spanning Tree : $\rightarrow$

Spanning Tree : $\rightarrow$ A tree in which we have N nodes & $N-1$ edges and all nodes are reachable from all other nodes is a Spanning Tree.

(graph)

Spanning Tree

The Minimum Spanning Tree is the ST with the least edge/weight sum :

edge sum = 18

edge sum = 16

MST

* Draw the MST for the following graph : $\rightarrow$

Total Nodes = 6

Total Edges in MST

$N-1$

$6-1=5$

MST sum $\Rightarrow$ 1+4+3+3+7 $\Rightarrow$ 16

MST sum $\Rightarrow$ 1+4+3+3+7 $\Rightarrow$ 16

2 Algorithms to find MST : $\rightarrow$

1) Prim's Algo

2) Kruskal's Algo

Disjoint Set Data Structure

Prim's Algorithm : $\rightarrow$

[MST sum, Edges]

start

0 1 2 3 4

1 2 3 4

MST = [(0, 2), (1, 2), (2, 3), (3, 4)]

visit 2 [] =

0 1 2 3 4

1 2 3 4

Minimum Spanning Tree

sum = [0+1+1+2+1]

0 0, 0, -1

2 1, 2, 0

1 1, 1, 2

2 2, 1, 0

3 2, 3, 2

1, 4, 3

Sum, node, parent

min-heap (PS)

* We will only consider an edge for the MST only if it has not been considered before.

* If it has already been included, it's discarded/ ignored.

* Only when we take a node in the MST sum, we mark it as visited.

* Principles of Prim's Algorithm.

* Number of Islands + Kruskal's Algo

* Dynamic Programming

Logic (Concepts)